

Retrieval Augmented Generation

RAG is a technique that enhances model's generation by retrieving the relevant information from external memory sources.

The external source can be internal database, previous user chats or the internet.

RAG is a solution for knowledge intensive task where all available knowledge cannot be input to your model.

With RAG, only the information most relevant to the query, as determined by the retriever, is retrieved & input into the model. "This helps hallucinations to some extent".

RAG Architecture:

A RAG has two components.

(1) Retrieval

(retrieves info from external memory resources)

(2) Generation

(generates the response based on the retrieved information)

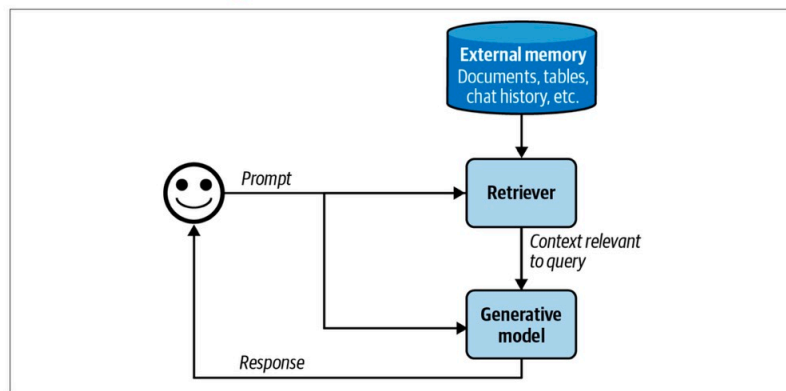


Figure 6-2. A basic RAG architecture.

In older days, the two components are trained together.

In today's RAGs, both are trained separately and many teams build RAGs using the off-the-shelf retrievers and models. However finetuning the whole RAG system end-end can improve the performance significantly.

The success of RAG depends on the quality of retriever

The retriever has two components: Indexing & Querying.

Indexing: It involves processing the data so that it can be quickly retrieved later.

Querying: Sending the query to retrieve data relevant to it.

Retrieval Algorithms:

Two commonly used retrieval mechanisms

- (i) Term based Retrieval
- (ii) Embedding based Retrieval

Term Based Retrieval

Term Based Retrieval is more like (Sparse Retrieval) where vectors have mostly 0s.

Term Based retrieval are most straightforward approach which finds the relevant documents using keywords (a.k.a lexical retrieval).

2 problems:

1.) Many documents may contain the keyword and there won't be sufficient context space to accommodate all the documents where the term is present.

The number of times the term present is called "Term Frequency". The heuristic to use TF to find docs with greatest _{TF}

2.) A prompt can be long and contains many terms. Some terms are more important than others. We want to focus on more informative terms.

The intuition that more the document has the term, less the information is.

$$\text{Term's importance} \propto \frac{1}{\text{no. of documents it appears in}} \quad (\text{IDF})$$

Elastic Search & BM25 (Best Matching 25) are two common term based retrieval solutions.

BM25 and its variances are still widely used against modern, more sophisticated retrieval algorithms (embedding based).

Embedding Based Retrieval

Term based computes the lexical relevance than semantic level. This results in retrieving information irrelevant to query.

Embedding based Retrieval aim to rank documents based on how closely they align with query. This approach is called semantic retrieval.

With embedded based retrieval, indexing has an extra function: converting original data chunks into embeddings. The database where the generated embedding stored are vector databases.

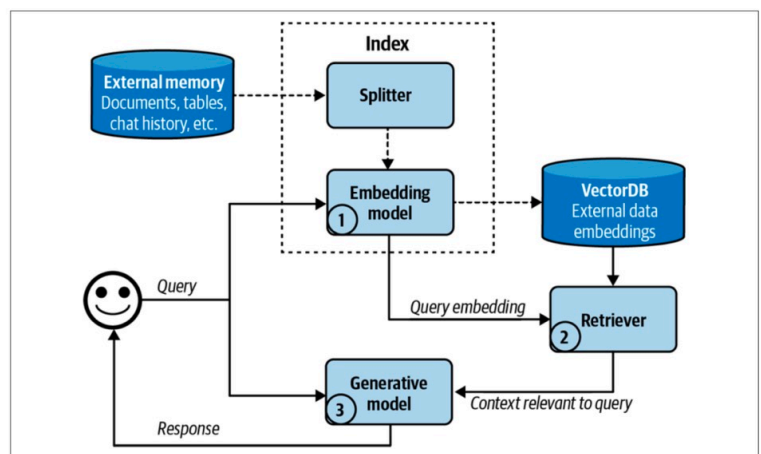


Figure 6-3. A high-level view of how an embedding-based, or semantic, retriever works.

Querying then consists of 2 steps:

Step 1: Convert the query into embedding using the same embedder used for indexing.

Step 2: Retriever: fetch k data chunks whose embeddings close to the query embedding, as determined by the retriever.

The number of chunks ' k ' depends on the use case, the generative model and the query.

In Real world, Retriever might work with reranker to rerank all the retrieved candidates & reduce latency issues.

Embedding based Retrieval introduces new component called Vector Databases: Storing is the easiest part of vector database. The hard part is vector search.

Given a query embedding, a vector database is responsible for finding vectors in the database that close to the query & returning them. Vectors have to be indexed and stored in a way that makes vector search fast & efficient.

Vector Search typically framed as k Nearest Neighbor (KNN) algorithm.

- (1.) Compute & compare similarity score between the query embedding & all the vectors in the database.
- (2.) Rank the vectors by similarity score
- (3.) Return k vectors with high similarity score.

This is computationally heavy & slow for large datasets.

For Large Dataset, we use Approximate Nearest Neighbor (ANN) for vector search. Already many algorithms & libraries implemented for vector search.

Libraries for Vector Search:

- (i) FAISS (Facebook AI Similarity Search)
- (ii) Google's scanN (Scalable Nearest Neighbor)
- (iii) HNSWlib (Hierarchical Navigable Small World)

In general, vector databases are organized into trees, nodes or graph. Vector search algorithms differ based on the heuristic they use to increase the likelihood to search the similar vector close to each other.

Significant Vector Search Algorithms:

(i) LSH (Locality-sensitive Hashing)

- * powerful & versatile algorithm
- * It involves hashing similar vectors into same buckets to speed up similarity search, trade some accuracy for efficiency.
- * used in FAISS & annoy.

(ii) Hierarchical Navigable Small World (HNSW)

* It constructs multilayer graph where each nodes represent vectors and edges connect the similar vectors allowing nearest neighbor searches by traversing graph edges.

* used in FAISS & Milvus

(iii) Product Quantization

Reduce the vectors in lower dimensional space into multiple subvectors. Then their distances are calculated which are much faster to work with.

Product Quantization is key component of FAISS & supported by all popular vector search algos.

(iv) Inverted File Index

IVF uses k means clustering to organize similar vectors into ^{same} clusters

During querying, the query embedding match against all cluster centroids for similarity search. The one cluster member becomes the candidates neighbors.

Product Quantization + IVF becomes the backbone of FAISS.

(v) (Approximate Nearest Neighbor Oh Yeah)

Annoy is a tree based approach.

It builds multiple binary trees, where each tree splits the vectors into clusters using random criteria. Eg: splitting the vectors into ^{two} branches by drawing a line randomly.

During the search, it traverses through ^{these} trees to find the nearest neighbor.

Comparison:

Text Based	Embedding Based
* fast with indexing & querying * outperform out of box but hard to improve * provide wrong docs due to term ambiguity. * Much cheaper than Embedding based	* slower compared to text based * Can outperform text based with finetuning * It focuses on semantic terms. * Embedding, vector storage & vector space can be expensive.

There are lot of Evaluation metrics to evaluate RAG frameworks. The most often are Context Precision & context Recall.

Context Precision/Context Relevance;

Out of all documents retrieved, what percentage is relevant to query?

Context Recall

Out of all documents that are relevant to query, what percentage is retrieved?

Context Precision is often supported in industries, since it is easy to compute by comparing retrieved documents to the query, which is done by AI Judge.

Context recall is big work where need to annotate each embedding in vector if its relevant or not where industries don't prefer.

If we care about ranking of retrieved documents
Eg: relevant docs should be ranked first.

Metrics for ranking are

- (i) Normalized Discounted Cumulative Gain (NDCG)
- (ii) Mean Average Precision (MAP)
- (iii) Mean Reciprocal Rank (MRR)

Note: The added latency for query embedding generation & vector search might be minimal compared to the total RAG latency. Even so, the added latency can impact the user experience.

With retrieval systems, you can make trade-off between indexing & querying.

The more detailed the index is, the more the answers are accurate but indexing process becomes slower & more memory consuming.

Eg: HSNW

detailed index & great accurate but faster sometimes.

But time consuming and more memory to build.

LSH

Simple index & less memory

But slower & less accurate queries.

For indexing & querying, the four metrics are considered

* Recall

* Query Per Second + more retrieval benchmarks

* Build time

* Index Size

Summary:

Quality of RAG is evaluated by both component-to-component & end-to-end. To do this, follow

- 1.) Evaluate the retrieval quality
- 2.) Evaluate the final RAG outputs
- 3.) Evaluate the embeddings (for embedding based)

Combining Retrieval Algorithms:

Hybrid Approach: 1.) Retrieve the docs by term based from the Vector DB
2.) Use expensive mechanisms & precise models to rank these candidates. This step is called reranking.

Different algorithms can be combined to form an ensemble Method. The algorithm for combining different ranking scores is called **reciprocal rank fusion (RRF)**

Retrieval Optimization:

Some tactics can help to increase the chance of relevant documents to be fetched. They are

- 1) Chunking Strategy
- 2) Reranking
- 3) Query rewriting
- 4) Contextual retrieval

1.) Chunking Strategy:

There is no universal chunk size (or) overlap size. You have to experiment & find what best suits for us.

The lesser are chunk sizes, the more wide ^{diverse} information can be retrieved. However, smaller chunk size might face loss of information. Eg: Two paragraphs where X is topic and the first part has X context but second paragraphs doesn't, At this situation, we might lose info.

2.) Reranking: (discussed in ensemble Retrieval Algo)

Documents can also be reranked based on time, giving more weight to recent data (Eg: Time sensitive apps, chats, news, stocks)

3.) Query rewriting:

Ex: User: When was the last time Joe John bought something from us?

AI: gives some answer

User: What about Prasanth?

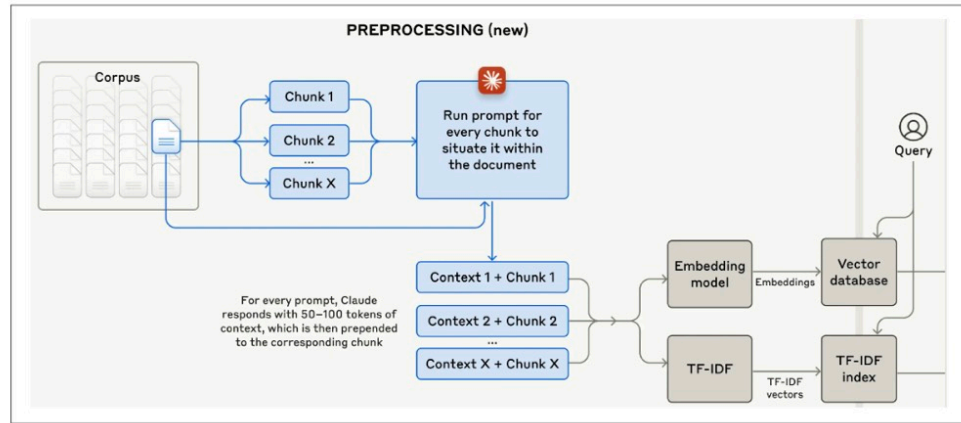
For this last question, we can again use LLM to rewrite query to "When was the last time prasanth bought something from us".

(4) Contextual Retrieval.

The idea is to augment chunk with relevant context for faster relevant chunk retrieval.

We can augment chunk with metadata like tags or keywords or title or summary.

Eg:



The generated context can be prepended to each chunk and then indexed by retrieval algorithms.

Evaluating Retrieval Solutions

Here are some key factors to keep in mind when evaluating a retrieval solution:

- What retrieval mechanisms does it support? Does it support hybrid search?
- If it's a vector database, what embedding models and vector search algorithms does it support?
- How scalable is it, both in terms of data storage and query traffic? Does it work for your traffic patterns?
- How long does it take to index your data? How much data can you process (such as add/delete) in bulk at once?
- What's its query latency for different retrieval algorithms?
- If it's a managed solution, what's its pricing structure? Is it based on the document/vector volume or on the query volume?

This list doesn't include the functionalities typically associated with enterprise solutions such as access control, compliance, data plane and control plane separation, etc.

Evaluation Metrics

(1) Context Precision

$$\text{precision@k} = \frac{\text{TP@k}}{\text{TP@k} + \text{FP@k}}$$

$$\text{Context Precision@k} = \frac{\sum_{k=1}^K (\text{Precision@k} \times r_k)}{\text{Total relevant items in top k}}$$

if score is higher, relevant results are ranked at top

if score is lower, relevant results are missing or ranked lower. Fix: implement reranking + query reformulation.

$$\text{Context Precision} = \frac{\text{Relevant Retrieved Context}}{\text{All retrieved Context}}$$

(2) Context Recall

$$\text{Context Recall} = \frac{\text{Relevant Retrieved Context}}{\text{All Groundtruth Context}}$$

(3) Response Relevancy:

here we find cosine similarity for the user query against questions generated from LLM response.

$$\text{Answer/Response Relevancy} = \frac{1}{N} \sum \text{CosineSim}(Q_{gi}, Q_u)$$

This metric tells us how the response directly answers the questions with clear & relevant information.

(4) Faithfulness:

here LLM response generated from user query & context given. From that response, we find claims using LLMs.

$$\text{Faithfulness} = \frac{\text{Supported claims}}{\text{Total claims}}$$

Note:

Response Relevancy checks if answer is being on topic or not.

Faithfulness checks being truthful to the source material.

BertScore: Evaluates how semantically similar the generated text to the referenced text using contextual embeddings.

Cons: Computationally extensive
Less interpretable than BLEU & ROUGE.

BLEU: (Bilingual Evaluation Understudy)

Measures n-gram precision between the candidate & referenced text.

Designed for machine translation.

Pros: * Simple, fast, widely used
* Good for machine translation

Cons:
* no semantic capture
* penalizes valid paraphrases with diff words

ROUGE (Recall Oriented Understudy for Gist Evaluation)

Measures n-gram recall overlap - how much of the reference content is covered by generated text.

Popular for summarization tasks.

ROUGE-n; n: n gram overlap

ROUGE-L; L - Longest Common Subsequence.

Perplexity: measure of how model "perplexed" or "confused" when predicting the next sequence in the word.

RAGIAS (RAG Assessment)

It is the RAG evaluation framework specifically designed for RAG pipelines.

Key Metrics:

- 1) **Context Relevance**: Whether the retrieved contexts are relevant.
- 2) **Context Precision**: Noise vs relevance
- 3) **Context Recall**: checks out of all relevant info, how much is retrieved (missing vs retrieved info)
- 4) **Answer Relevance**: If the generated answer is related to query or not
- 5) **Answer Correctness**: Is the answer factually correct?
- 6) **Faithfulness**: If the answer is grounded in generated context? (hallucination measure)

DeepEval: for broader testing and evaluation framework for LLMs, RAG and agentic workflows, with more emphasis on unit testing, regression testing, automatic scoring, CI/CD integrations & custom evaluators.

Key features: 1.) LLM as a judge evaluations 2.) Truthfulness & hallucination scoring 3.) Relevance Scoring 4.) Toxicity / Bias checks 5.) End to end RAG test cases.

Detect & Evaluate the hallucinations:

Three Detection Methods:

- A) ^{context} Grounded Evaluation. B) Truthfulness C) Consistency

A) Grounding: Does the output match with retrieved Context?

Methods: RAGAs faithfulness (0-1), DeepEval Hallucination evaluator, Embedding similarity (Answer $\leftrightarrow$ chunks)

If the answer says something that does not present in the context $\rightarrow$ hallucination

B) Truthfulness (External verification)

If the external reality contradicts the answer $\rightarrow$ hallucination

Methods: * API verification (finance, weather)

* Fast Checker Model (FEVER, TruthfulQA prompts)

* Cross Model Disagreements (Claude vs ChatGPT)

C) Consistency:

If the model contradicts itself.

Methods:

Self consistency decoding (multiple generations $\rightarrow$ compare)

Cross LLM verification